



सो रिसेंटली हमने जावास्क्रिप्ट ...  
TXT 1.54MB

explain everything properly in english

## React JS Complete Course - Detailed English Explanation

This is a comprehensive Hindi transcript of a React JS tutorial by **Sarthak Sharma** from **Sheriyans Coding School**. Let me break down everything explained in this massive video/course.

### 1. INTRODUCTION & CONTEXT

The instructor explains that after his "JavaScript Learn Everything" video, viewers demanded a React JS equivalent. This video covers everything needed to become a **job-ready, industry-level React developer**.

**Key promise:** The video includes:

- Multiple projects
- Real-life use cases
- Hidden details not covered in other tutorials
- Broken into chapters for easy consumption

### 2. WHAT IS REACT JS?

**Definition:** React JS is a **JavaScript library** used to build UIs (User Interfaces), specifically for complex frontends.

**History:**

- Created by **Meta (formerly Facebook)**
- Released in **2013**
- One of the most in-demand skills in 2025 despite AI advancements

# The Story of Why React Was Needed

## The Problem (Facebook circa 2010-2011):

When Facebook's user base grew, they faced a major issue:

- If someone sent you a message, the notification count (e.g., "3") wouldn't update until you **reloaded the entire page**
- Same for friend requests, likes, notifications
- Every small update required a **full page reload**
- This destroyed user experience

## Traditional JavaScript approach:

- With plain HTML/CSS/JS, every change requires updating everything
- No efficient way to update just one part of the UI

## React's Solution:

- **Component-Based Architecture** - Break the entire website into separate, independent chunks/parts
- Example: On a website, the navbar is one component, notification count is another component, sidebar is another, etc.
- When a notification arrives, **only that small component re-renders** - not the whole page
- This creates a "buttery smooth" feel

## YouTube Example:

- When you search for "Sheriyans Coding School" and click a video, the page changes
- But the search bar text ("Sheriyans Coding School") **stays the same** - it doesn't re-render
- Only the content below changes
- This is the benefit of React - only necessary parts update

## 3. JAVASCRIPT PREREQUISITES

Before learning React, you should know:

- **Variables** (let, const, var)
- **Loops**
- **Conditionals** (if/else)
- **Data types**
- **Functions and Arrow Functions**
- **Return statements**
- **Arrays** - map, filter, reduce, forEach
- **Objects** and Object Literals
- **ES6 features** (let, const, arrow functions, destructuring)
- **Asynchronous JavaScript** - Promises, Fetch API, async/await

The instructor says if you don't know these, don't worry - he'll explain as needed.

## 4. SETTING UP THE DEVELOPMENT ENVIRONMENT

Tools used:

- **VS Code** (code editor) - preferred over Cursor, Windsurf, Trae, Sublime Text
- **Live Server** extension for basic HTML files
- **Dark Reader** extension for dark theme (for eye protection)

**Basic JavaScript DOM Example:**

```
javascript

// Creating an element in vanilla JS
const h1 = document.createElement("h1");
h1.innerHTML = "Hello from JS";
document.body.appendChild(h1);
```

This creates an H1 element with "Hello from JS" and adds it to the body.

## 5. LIBRARY VS FRAMEWORK

**Library:**

- Used for a **specific feature**
- Examples: GSAP (animations), Locomotive Scroll (smooth scrolling), React (UI)
- You buy a plot and build your house however you want
- More customization, more control

### Framework:

- Everything comes **built-in**
- Examples: Angular, Next.js, Remix
- You buy a ready-made flat - can't change much
- Less customization, but more structure

**React is a library** - only handles UI. React team has now stopped recommending `create-react-app` and suggests using React with frameworks like Next.js or Remix.

## 6. IMPORT/EXPORT IN JAVASCRIPT

### Default Export

- A file exports **only one thing** by default
- When importing, you can name it **anything**

```
javascript
```

```
// app.js
```

```
const arr = [10, 20, 30];
```

```
export default arr;
```

```
// script.js
```

```
import array from './app.js'; // Can name it anything
```

### Named Export

- Multiple things can be exported
- Must use **curly braces** when importing
- Must use the **exact name**

```
javascript
```

```
// app.js
export const user = "Sarthak";
export const age = 89;

// script.js
import { user, age } from './app.js';
```

**Important:** Type must be `module` in the script tag:

```
html
```

```
<script type="module" src="script.js"></script>
```

## 7. REAL DOM VS VIRTUAL DOM

### Real DOM

- The actual HTML tree structure
- When you update something, the **entire DOM re-renders**
- Example: A register with 1000 entries - changing one entry requires rewriting everything

### Virtual DOM

- React creates a **copy of the Real DOM**
- When changes happen, React **compares** the virtual DOM with the real DOM
- Only the **changed parts** are updated
- This is why React is fast and efficient

#### Real-world example:

- Instagram notification count: When a 5th message arrives, only the count updates - not the whole page
- YouTube: When you navigate, the search bar stays the same

## 8. JSX (JavaScript XML)

**Definition:** A combination of HTML and JavaScript.

**Without JSX (vanilla JS):**

javascript

```
const h1 = document.createElement("h1");  
h1.innerHTML = "Hello";
```

**With JSX:**

jsx

```
const h1 = <h1>Hello</h1>;
```

**Calling functions in JSX:**

jsx

```
// Both work  
{ABC()}  
<ABC />
```

**Rules:**

- Can't return two things at once from a function
- Use a **div wrapper** or **Fragment** ( <>...</> )
- Use `className` instead of `class` (class is a reserved keyword)
- Use `htmlFor` instead of `for`

## 9. CREATING A REACT PROJECT WITH VITE

**Command:**

bash

```
npm create vite
```

**Steps:**

1. Enter project name
2. Select React
3. Select JavaScript (or TypeScript)
4. `cd project-name`
5. `npm install`
6. `npm run dev`

**Folder Structure:**

text

```

project/
├─ node_modules/    (dependencies - don't touch)
├─ public/           (static assets - images, fonts)
├─ src/
│   ├─ assets/       (SVGs, frequently used images)
│   ├─ App.jsx        (main component)
│   ├─ main.jsx       (entry point - renders App)
│   ├─ index.css      (global styles)
│   └─ App.css        (component styles)
├─ index.html        (HTML boilerplate with root div)
├─ package.json      (configurations, dependencies)
├─ vite.config.js    (Vite configurations)
├─ .eslintrc.js      (ESLint rules)
├─ .gitignore        (files not to upload to Git)
└─ README.md         (project description)

```

**The Flow:**

1. `index.html` has `<div id="root"></div>`
2. `main.jsx` selects the root div and renders `<App />` into it
3. `App.jsx` is the main component that contains everything

**Hierarchy:** `index.html` → `main.jsx` → `App.jsx` → other components

## 10. COMPONENT-BASED ARCHITECTURE

**What is a component?**

- A small piece of code that serves a specific feature
- Examples: Navbar, Card, Button, Footer

### Benefits:

- Reusability - write once, use multiple times
- Easy updates - change one component without affecting others
- Better performance - only that component re-renders

### Example:

- Instagram feed: Same card structure, different data
- YouTube: Same video card structure, different content
- Story feature: Same format, different user data

### How to create:

1. Create a `components` folder in `src`
2. Create `Card.jsx` file
3. Write function and export it
4. Import and use in parent component

jsx

```
// components/Card.jsx
const Card = () => {
  return (
    <div className="card">
      <h1>Hello</h1>
    </div>
  );
};
export default Card;

// App.jsx
import Card from './components/Card';
const App = () => {
  return (
    <div>
      <Card />
      <Card />
      <Card />
    </div>
  );
};
```

```
);  
};
```

## 11. PROPS (Properties)

### What are Props?

- Data passed from **parent to child**
- Always flow **top to bottom** (never bottom to top)
- Received as an **object**

### Passing props:

```
jsx  
  
<Card user="Sarthak" age={21} />
```

### Receiving props:

```
jsx  
  
const Card = (props) => {  
  console.log(props); // { user: "Sarthak", age: 21 }  
  return <h1>{props.user}</h1>;  
};
```

### Important:

- Strings: `user="Sarthak"` (direct)
- Numbers/Arrays/Objects: `age={21}` (curly braces)
- Can rename: `const Card = (chacha) => { chacha.user }`

### Props Drilling:

- Passing props through multiple levels
- App → Section → Page → RightContent → Card
- Each level must pass the prop down
- Problem: Middle components don't even need the data but must pass it

## 12. MAP METHOD FOR ARRAYS OF OBJECTS

### Example:

jsx

```
const users = [  
  { name: "Sarthak", age: 30, image: "..."},  
  { name: "Harsh", age: 36, image: "..."},  
  { name: "Aman", age: 20, image: "..."}  
];  
  
// In component  
{users.map((user, idx) => (  
  <Card key={idx} user={user.name} age={user.age} />  
))}
```

### Key Points:

- `map` returns a new array
- Always pass a `key` prop (unique identifier)
- Key helps React identify which elements changed
- Better to wrap in a div and pass key to the div (not directly to component)

## 13. CSS IN REACT

### 1. Global CSS

- `index.css` - applies to everything
- Good for small projects
- Problem: Hard to manage in large projects

### 2. Component-Specific CSS

- Separate CSS files per component
- `Button.css`, `Header.css`
- Import in respective component files

- Problem: Class name collisions

### 3. CSS Modules

- File naming: `Button.module.css`
- Import as `styles` object
- Use `className={styles.btn}`
- Classes get unique names (hashed)
- Prevents collisions

jsx

```
// Button.module.css
.btn { background: red; }

// Button.jsx
import styles from './Button.module.css';
<button className={styles.btn}>Click</button>
```

### 4. Tailwind CSS (Utility-First)

- Not a component library - utility-first
- Classes like `bg-red-600`, `text-white`, `px-5`, `py-2`
- Modern approach
- Install via npm

#### Tailwind Setup:

bash

```
npm install tailwindcss @tailwindcss/vite
```

Then configure `vite.config.js` and import in `index.css`.

## 14. FEATURE-BASED VS ATOMIC DESIGN

#### Feature-Based:

- Break by features/pages
- Entire navbar = one component
- Entire card section = one component
- Each page = one component

### Atomic Design:

- Break into smallest pieces
- Logo = separate component
- Nav links = separate component
- Button = separate component
- Heading = separate component
- Reuse everywhere

**Best Practice:** Combination of both.

## 15. FUNCTION CALLING IN REACT

### Event Handling:

jsx

```
const btnClicked = () => {  
  console.log("Button clicked");  
};
```

```
<button onClick={btnClicked}>Click</button>  
// NOT onClick={btnClicked()} - this would call immediately
```

### With parameters:

jsx

```
<button onClick={() => btnClicked("param")}>Click</button>
```

### Multiple events:

- onClick - click

- `onDoubleClick` - double click
- `onMouseEnter` - mouse enter
- `onMouseLeave` - mouse leave
- `onChange` - input change (for form inputs)
- `onSubmit` - form submission
- `onScroll` / `onWheel` - scrolling

### Event object:

jsx

```
const handleChange = (e) => {  
  console.log(e.target.value); // Current input value  
  console.log(e.target); // The target element  
};
```

### Prevent default:

jsx

```
const handleSubmit = (e) => {  
  e.preventDefault(); // Prevents page reload  
};
```

## 16. HOOKS - INTRODUCTION

### What are Hooks?

- Special functions in React
- Help manage state, side effects, context, etc.
- Class components are now obsolete

### Types of Hooks:

| Hook                  | Purpose      |
|-----------------------|--------------|
| <code>useState</code> | Manage state |

| Hook                     | Purpose                                                            |
|--------------------------|--------------------------------------------------------------------|
| <code>useEffect</code>   | Handle side effects (API calls, DOM manipulation, event listeners) |
| <code>useContext</code>  | Manage global context                                              |
| <code>useReducer</code>  | Complex state management (like Redux)                              |
| <code>useRef</code>      | Reference DOM elements, hold mutable values                        |
| <code>useMemo</code>     | Optimize performance (memoization)                                 |
| <code>useCallback</code> | Optimize functions (memoization)                                   |

## 17. USESTATE HOOK

### Why needed?

- Variables don't update the UI when changed
- React needs to be told to update the UI

### Without `useState` (wrong):

jsx

```
let a = 20;  
const changeA = () => { a = 30; }; // UI doesn't update
```

### With `useState`:

jsx

```
const [a, setA] = useState(20);  
const changeA = () => { setA(30); }; // UI updates
```

### Syntax:

jsx

```
const [value, setValue] = useState(initialValue);
```

- `value` - read-only current value
- `set` - function to update value

## Counter Example:

jsx

```
const [num, setNum] = useState(0);

const increase = () => setNum(num + 1);
const decrease = () => setNum(num - 1);
const jumpFive = () => setNum(num + 5);

<button onClick={increase}>Increase</button>
<button onClick={decrease}>Decrease</button>
<button onClick={jumpFive}>Jump by 5</button>
<h1>{num}</h1>
```

## Key Points:

- State updates are **asynchronous**
- `console.log` after `setState` shows old value
- React **batches** updates for performance
- Don't mutate state directly - always use setter

## Updating Objects/Arrays

### Wrong:

jsx

```
const [user, setUser] = useState({ name: "Sarthak", age: 20 });
user.name = "Aman"; // No re-render
```

### Correct:

jsx

```
const newUser = { ...user, name: "Aman" };
setUser(newUser);
// Or
setUser(prev => ({ ...prev, name: "Aman" }));
```

## For Arrays:

jsx

```
const newArr = [...arr, 99];
setArr(newArr);
```

## Batch Updates:

jsx

```
// Wrong - only increments by 1
setNum(num + 1);
setNum(num + 1);
setNum(num + 1);

// Correct - increments by 3
setNum(prev => prev + 1);
setNum(prev => prev + 1);
setNum(prev => prev + 1);
```

# 18. FORM HANDLING & TWO-WAY BINDING

## Form Submission:

jsx

```
const submitHandler = (e) => {
  e.preventDefault(); // Prevents page reload
  console.log("Form submitted");
};

<form onSubmit={submitHandler}>
  ...
</form>
```

## Two-Way Binding:

- Input value is controlled by React state
- Changes in input update state
- State updates input value

jsx

```
const [title, setTitle] = useState("");

<input
  value={title}
  onChange={(e) => setTitle(e.target.value)}
  placeholder="Enter title"
/>
```

## Why needed?

- Direct DOM manipulation is not React's way
- React should control everything
- Enables controlled components

## Multiple inputs:

jsx

```
const [title, setTitle] = useState("");
const [details, setDetails] = useState("");

<input value={title} onChange={(e) => setTitle(e.target.value)} />
<textarea value={details} onChange={(e) => setDetails(e.target.value)} />
```

## Form submission with data:

jsx

```
const submitHandler = (e) => {
  e.preventDefault();
  console.log(title, details);
  setTitle("");
  setDetails("");
};
```

# 19. LOCAL STORAGE

## What is it?

- Browser's memory
- Stores data as **key-value pairs**
- Value must be a **string**
- Data persists even after closing browser

### Methods:

```
javascript

// Set
localStorage.setItem("user", "Sarthak");

// Get
const user = localStorage.getItem("user");

// Remove specific
localStorage.removeItem("user");

// Clear all
localStorage.clear();
```

### For objects/arrays:

```
javascript

// Store
localStorage.setItem("user", JSON.stringify(userObject));

// Retrieve
const user = JSON.parse(localStorage.getItem("user"));
```

### Session Storage vs Local Storage:

- **Session Storage:** Data persists until browser is closed
- **Local Storage:** Data persists forever (until manually cleared)

## 20. API HANDLING

### The Concept

**Frontend** = Restaurant interior, ambiance, menu

**Backend** = Kitchen where food is prepared

**API** = Waiter connecting both

### Types of API requests:

- **GET** - Fetch/retrieve data
- **POST** - Send data to backend
- **PATCH/PUT** - Update data
- **DELETE** - Delete data

## Fetch API

jsx

```
const getData = async () => {  
  const response = await fetch("https://jsonplaceholder.typicode.com/todos/1");  
  const data = await response.json();  
  console.log(data);  
};
```

## Axios (Preferred)

bash

```
npm install axios
```

jsx

```
import axios from "axios";  
  
const getData = async () => {  
  const response = await axios.get("https://api.example.com/data");  
  console.log(response.data);  
};
```

### Key Points:

- Response is a **Promise**
- Use `async/await` to handle

- `response.json()` is also async (for fetch)
- Axios gives data directly in `response.data`
- JSON = JavaScript Object Notation (format for data transfer)

## Storing API Data in State

jsx

```
const [userData, setUserData] = useState([]);

const getData = async () => {
  const response = await axios.get("https://api.example.com/users");
  setUserData(response.data);
};

// Render
{userData.map((user, idx) => (
  <div key={idx}>
    <img src={user.download_url} alt="" />
    <h2>{user.author}</h2>
  </div>
))}
```

## Pagination

jsx

```
const [index, setIndex] = useState(1);
const [data, setData] = useState([]);

const getData = async () => {
  const response = await axios.get(`https://api.example.com/data?page=${index}&limit=10`);
  setData(response.data);
};

const prev = () => {
  if (index > 1) {
    setIndex(index - 1);
  }
};

const next = () => {
```

```
    setIndex(index + 1);  
  };
```

## Infinite Scrolling

- Load more data as user scrolls
- Trigger API call on scroll
- Append new data to existing

## 21. USEEFFECT HOOK

### What is it?

- Handles **side effects** in React
- Side effects = API calls, DOM manipulation, timers, event listeners
- Runs after the render process

### Concept:

- React's main process (rendering) = main road with a big truck
- Side effects = side lane / footpath
- `useEffect` provides a side lane for parallel processes

### Basic Usage:

```
jsx  
  
useEffect(() => {  
  console.log("Running");  
});  
// Runs on EVERY render (wrong usage)
```

### With Empty Dependency Array:

```
jsx  
  
useEffect(() => {  
  console.log("Runs once");  
});
```

```
}, []);  
// Runs only ONCE on mount
```

## With Dependencies:

```
jsx  
  
useEffect(() => {  
  console.log("A changed");  
}, [a]);  
// Runs when 'a' changes
```

## Multiple dependencies:

```
jsx  
  
useEffect(() => {  
  // Runs when a OR b changes  
}, [a, b]);
```

## Cleanup:

```
jsx  
  
useEffect(() => {  
  // Setup  
  return () => {  
    // Cleanup (runs on unmount)  
  };  
}, []);
```

## Common Use Cases:

- API calls on component mount
- Event listeners
- Timers (setInterval, setTimeout)
- Local storage sync

## Example - API call on mount:

```
jsx
```

```
useEffect(() => {  
  getData();  
}, []);
```

## 22. REACT ROUTER DOM

### What is Routing?

- Navigating between different pages
- React doesn't have built-in routing
- Need **react-router-dom** library

### Installation:

```
bash
```

```
npm install react-router-dom
```

### Types of Routers:

- **BrowserRouter** - Most common, uses HTML5 History API
- **HashRouter** - URL has #, used for static sites
- **MemoryRouter** - Stores in memory (React Native)
- **StaticRouter** - Used in Next.js/Remix

### Basic Setup:

```
jsx
```

```
// main.jsx  
import { BrowserRouter } from 'react-router-dom';  
  
<BrowserRouter>  
  <App />  
</BrowserRouter>
```

```
jsx
```

```
// App.jsx
import { Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';

<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
</Routes>
```

## Navigation with Link:

jsx

```
import { Link } from 'react-router-dom';

<Link to="/">Home</Link>
<Link to="/about">About</Link>
```

## Not Found Page:

jsx

```
<Route path="*" element={<NotFound />} />
```

## Nested Routes:

jsx

```
<Route path="/product" element={<Product />>
  <Route path="men" element={<Men />} />
  <Route path="women" element={<Women />} />
</Route>
```

*// In Product component*

```
import { Outlet } from 'react-router-dom';

<div>
  <h1>Product Page</h1>
  <Outlet /> {/* Child routes render here */}
</div>
```

## Dynamic Routes:

jsx

```
<Route path="/courses/:courseId" element={<CourseDetail />} />
```

```
// In CourseDetail
```

```
import { useParams } from 'react-router-dom';
```

```
const { courseId } = useParams();
```

```
console.log(courseId); // Value from URL
```

## Navigate Programmatically:

jsx

```
import { useNavigate } from 'react-router-dom';
```

```
const navigate = useNavigate();
```

```
// Go to home
```

```
navigate("/");
```

```
// Go back
```

```
navigate(-1);
```

```
// Go forward
```

```
navigate(1);
```

## 23. CONTEXT API

### Problem: Props Drilling

- Data must be passed through every level
- App → Navbar → Sidebar → UserProfile → Username
- Middle components don't even need the data

### Solution: Data Centralization

- Like writing password on a wall for everyone to access
- Data stored in one place, accessible by all

### Three Steps:

## 1. Create Context:

```
jsx

import { createContext } from 'react';
export const ThemeDataContext = createContext();
```

## 2. Provide Data:

```
jsx

<ThemeDataContext.Provider value={theme}>
  <App />
</ThemeDataContext.Provider>
```

## 3. Use Data:

```
jsx

import { useContext } from 'react';
import { ThemeDataContext } from '../context/ThemeContext';

const data = useContext(ThemeDataContext);
```

## Full Example:

```
jsx

// ThemeContext.js
import { createContext } from 'react';
export const ThemeDataContext = createContext();

// App.jsx
import { ThemeDataContext } from '../context/ThemeContext';
import { useState } from 'react';

const [theme, setTheme] = useState("light");

<ThemeDataContext.Provider value={[theme, setTheme]}>
  <Navbar />
</ThemeDataContext.Provider>

// Button.jsx
import { useContext } from 'react';
import { ThemeDataContext } from '../context/ThemeContext';
```

```
const [theme, setTheme] = useContext(ThemeDataContext);

<button onClick={() => setTheme("dark")}>
  Change Theme
</button>
```

## Children Concept:

jsx

```
const Navbar = (props) => {
  console.log(props.children); // Children elements
  return <div>{props.children}</div>;
};

<Navbar>
  <h2>Hello</h2>
  <p>World</p>
</Navbar>
```

## 24. PROJECTS BUILT IN THE COURSE

1. **Cards Project** - User cards with props
2. **Job Listings** - Array of objects with map
3. **Website UI Project** - Digital banking platform with sections
4. **Notes App** - Form handling, two-way binding, delete functionality
5. **Gallery Application** - API handling, pagination, loading states

## 25. KEY TAKEAWAYS

1. **React is a library** for building UIs
2. **Component-based architecture** enables reusability and performance
3. **Virtual DOM** makes React fast
4. **Props** flow from parent to child only
5. **useState** manages state and triggers re-renders

6. **useEffect** handles side effects
7. **Context API** solves props drilling
8. **React Router** enables navigation
9. **API handling** via fetch or axios
10. **Practice is essential** - build projects to solidify concepts

## 26. IMPORTANT QUOTES FROM THE INSTRUCTOR

"React JS is basically a JavaScript library which is used to make UI."

"Props data always flows from top to bottom, never bottom to top."

"useEffect is one of the most powerful hooks but also one of the most confusing."

"Don't mutate state directly - always use the setter function."

"The best way to learn is by building projects."

"What is your 49.5?" (Referencing the movie Bhaag Milkha Bhaag - asking viewers about their ultimate goal)

This comprehensive explanation covers every major topic from the transcript. The course is designed to take someone from zero React knowledge to job-ready level, with practical projects and real-world examples throughout.

explain everything again deeply with proper explanation

## React JS Complete Course — Deep, Detailed English Explanation

This is a full, structured, and deeply explained breakdown of the entire Hindi transcript (a massive React JS course by **Sarthak Sharma** from **Sheriyans Coding School**). I'll explain every concept step by step with proper reasoning, analogies, code, and context.

## PART 1 — INTRODUCTION & CONTEXT

## 1.1 Why This Video Exists

The instructor previously uploaded a video titled **"JavaScript Learn Everything."** It received massive support, and almost every comment asked for a React JS equivalent: **"React JS Learn Everything laao"** (bring React JS Learn Everything).

So this video is the answer. It's not just a tutorial — it's a **complete course** designed to make you a **job-ready, industry-level React developer**.

## 1.2 What Makes This Different

Other tutorials teach:

- What is React?
- What are props?
- What are components?

But they **forget to teach** what a real developer actually needs:

- How work happens in real life
- Real-life use cases
- Hidden details about React
- Multiple projects
- Industry-level practices

This course focuses on exactly that.

## 1.3 How to Consume This Course

- The video is **huge**, broken into **small chapters**
- Recommendation: **Watch one chapter per day**
- Build the projects he demonstrates
- If you do this, he **guarantees** you'll learn React

"Main hoon aapka dost, aapka instructor Sarthak Sharma. Aap dekh rahe hain Sheriyans Coding School."

# PART 2 — WHAT IS REACT JS?

## 2.1 Simple Definition

React JS is a JavaScript library used to build UIs (User Interfaces).

In simple language: It builds the **frontend** — the part of the website you see.

You might ask: "But HTML, CSS, and JS also build frontends. Why React?"

Answer: React builds **complex frontends** much more easily.

## 2.2 History of React

| Detail         | Info                                                   |
|----------------|--------------------------------------------------------|
| Created by     | Meta (formerly Facebook)                               |
| Year           | 2013                                                   |
| Type           | JavaScript <b>Library</b> (not framework)              |
| Creator        | <b>Jordan Walke</b> (Facebook engineer)                |
| Current Demand | One of the most in-demand skills in 2025, even with AI |

## 2.3 The Story — Why React Was Needed

### The Problem at Facebook (2010–2011)

Facebook was growing rapidly. Users increased. And a **major problem** appeared.

#### Scenario 1 — Messages Count:

- You're scrolling Facebook. The Messages icon shows "3".
- Someone sends you a new message.
- The count should immediately become "4".
- **But it doesn't update until you reload the entire page.**

#### Scenario 2 — Likes/Notifications:

- Someone likes your post.
- You should get a notification.
- **But the notification count only updates after a full page reload.**

### Scenario 3 — Friend Requests:

- Someone sends you a friend request.
- The count should update instantly.
- **But you must reload the page to see it.**

## Why Was This Happening?

With plain HTML + CSS + JavaScript:

- The entire page is a **static site**
- Every small change requires updating **everything**
- There's no efficient way to update just one part

## The User Experience Problem

Think of it like this:

- Imagine a register with 1000 entries (names, phone numbers, addresses)
- Someone says: "Sarthak bhaiya's number is wrong — fix it"
- **You have to rewrite the entire register** just to fix one entry

That's how painful plain DOM manipulation was.

## React's Solution

React said:

"We'll move everything to **Component-Based Architecture**."

### What is Component-Based Architecture?

Break the entire website into **separate chunks / parts**:

- Navbar = one component
- Notification count = one component

- Sidebar = one component
- Feed = one component
- Each post = one component
- Each button = one component

Now when something changes:

- **Only that specific component re-renders**
- Nothing else re-renders
- The website feels **buttery smooth**

## Real Example — [Sheriyans.com](https://sheriyaans.com)

If you look at [Sheriyans.com](https://sheriyaans.com) built in React:

- The notification count is a **separate component**
- When a notification arrives, **only that small part** re-renders
- The rest of the page stays exactly the same
- The website feels extremely smooth

## YouTube Example (Even Though YouTube Isn't React)

Imagine you're on YouTube:

1. You search "Sheriyans Coding School"
2. You click a video → **page changes**
3. But notice: **the search bar text stays the same**
4. You click another video → **page changes again**
5. But the search bar **still stays the same**

Why? Because that part **doesn't need to re-render**.

This is called an **SPA (Single Page Application)** — and React is perfect for building these.

## PART 3 — JAVASCRIPT PREREQUISITES

Before React, you should know JavaScript. The instructor lists specific topics:

## 3.1 Basics

- **Variables** (let, const, var)
- **Loops**
- **Conditionals** (if/else)
- **Data types**

## 3.2 Functions

- **Functions**
- **Arrow functions**
- **Return statements**

## 3.3 Arrays

- **What are arrays**
- **Array methods:**
  - `map` — transform each element
  - `filter` — select certain elements
  - `reduce` — combine into one value
  - `forEach` — loop through

## 3.4 Objects

- **Object literals**
- **How to use objects**
- **Destructuring**

## 3.5 ES6 Features

- `let` and `const`
- **Arrow functions**
- **Template literals**

- Destructuring
- Spread operator
- Modules (import/export)

## 3.6 Asynchronous JavaScript

- Promises
- Fetch API
- `async/await`

## 3.7 What If You Don't Know These?

The instructor says:

"If you don't know these, no issue. I'll explain everything as needed. Don't worry — keep going."

# PART 4 — SETTING UP THE ENVIRONMENT

## 4.1 Tools

**VS Code** — the code editor used throughout. Other options:

- Cursor
- Windsurf
- Trae
- Sublime Text

He prefers **VS Code**.

**Live Server** — VS Code extension to run HTML files.

**Dark Reader** — for dark theme (eye protection).

## 4.2 Basic JavaScript DOM Refresher

Before showing React, he demonstrates how plain JS creates elements:

```
javascript

// Create an H1 element
const h1 = document.createElement("h1");

// Set its content
h1.innerHTML = "Hello from JS";

// Add to body
document.body.appendChild(h1);
```

### Explanation:

- `document.createElement("h1")` — creates an H1 element
- `h1.innerHTML = "..."` — puts text inside
- `document.body.appendChild(h1)` — adds it to the page

This is **manual, verbose, and painful** in large apps. React will change this.

## PART 5 — LIBRARY VS FRAMEWORK

### 5.1 What Is a Library?

A library is used for a **specific feature**.

#### Examples:

- **GSAP** — animation library
- **Locomotive Scroll / Lenis** — smooth scrolling
- **React JS** — building UI

#### Analogy — Plot vs Flat:

Imagine you want a house.

- **Library = You buy a plot**
  - You build however you want
  - Bathroom as big as you want
  - Full control, full customization

- More work, but more freedom
- **Framework = You buy a ready flat**
  - Everything is already built
  - You can't move walls
  - Less customization
  - Less work, but less freedom

## 5.2 What Is a Framework?

A framework gives you **everything built-in**.

**Examples:**

- **Angular**
- **Next.js**
- **Remix**

## 5.3 Key Differences

| Library            | Framework               |
|--------------------|-------------------------|
| Specific feature   | Complete architecture   |
| You control flow   | Framework controls flow |
| More customization | Less customization      |
| More setup work    | Less setup work         |
| React, GSAP, Lenis | Angular, Next.js, Remix |

## 5.4 React's Position

React is a **library**. It only handles UI.

The React team has now **stopped recommending** `create-react-app`. They suggest using React with frameworks (Next.js, Remix) for larger apps. But for learning, we build React projects directly.

## PART 6 — IMPORT / EXPORT IN JAVASCRIPT

This is an **ES6 feature** — critical for React.

### 6.1 Why Needed?

To split code across **multiple files** and share data between them.

### 6.2 Default Export

A file exports **only ONE thing** as default.

```
javascript

// app.js
const arr = [10, 20, 30];
export default arr;

// script.js
import array from './app.js'; // Can name it anything
```

**Key Point:** Since only one thing is exported, you can name it **whatever you want** during import.

### 6.3 Named Export

Multiple things can be exported with names.

```
javascript

// app.js
export const user = "Sarthak";
export const age = 89;

// script.js
import { user, age } from './app.js';
```

**Key Point:** Must use **curly braces** and the **exact name**.

### 6.4 Combining Both

javascript

```
// app.js
const user = "Sarthak";
export const age = 89;
export const skills = ["Frontend", "UI/UX"];
export default user;

// script.js
import naam, { age, skills } from './app.js';
```

- naam — default export (any name)
- age, skills — named exports (exact names)

## 6.5 The Tricky Concept

javascript

```
// app.js
const user = "Sarthak"; // default export
export const age = 89; // named export

// If you import like this:
import { age } from './app.js'; // age = 89
```

But:

javascript

```
import naam from './app.js'; // naam = "Sarthak"
```

### Important Rule:

javascript

```
// app.js
export default user; // user = "Sarthak"

// script.js
import { age } from './app.js'; // Looks for named export "age" → 89
import naam from './app.js'; // gets default export → "Sarthak"
```

**The trick:** If you write `import age from './app.js'`, you get the **default export**, not the named one. This causes bugs if you're not careful.

## 6.6 Module Script

To use import/export in plain HTML:

html

```
<script type="module" src="script.js"></script>
```

`type="module"` enables:

- Import/export
- Strict mode
- Other ES6 features

# PART 7 — REAL DOM VS VIRTUAL DOM

## 7.1 Real DOM

The **Real DOM** is the actual HTML tree structure in the browser.

html

```
<html>
  <head>
    <title>Page</title>
  </head>
  <body>
    <h1>This is my website</h1>
    <button>Change</button>
  </body>
</html>
```

Every tag becomes a node in a tree.

## The Problem With Real DOM

When you change **one thing** with plain JavaScript:

- The **entire DOM re-renders**
- This is slow and inefficient

### Analogy — The 1000-Entry Register:

Imagine a register with 1000 entries — names, phones, addresses.

Someone says: "Sarthak bhaiya ka number galat hai, last me 9 ki jagah 8 hona tha."

To fix **one entry**, you must:

- Open the register
- Find the entry
- Rewrite everything on the page
- Because changing one entry "breaks" the format

Same with Real DOM:

- Change "This is my website" to "This is my webpage"
- The **entire DOM re-renders**
- Even though you only changed 8 words

## The Facebook Problem Again

- You're scrolling Facebook
- A notification arrives
- Facebook's count should update to "1"
- But because of Real DOM, it **reloads the entire page**
- Bad user experience

## 7.2 Virtual DOM

React introduced the **Virtual DOM**.

**How it works:**

1. React creates a **copy of the Real DOM**

2. When changes occur, React updates the **Virtual DOM first**
3. React **compares** the Virtual DOM with the Real DOM
4. Only the **diff (difference)** is applied to the Real DOM

## Analogy

- Real DOM = the actual building
- Virtual DOM = a blueprint copy
- When you want to change a wall color:
  - You update the blueprint first
  - You compare old vs new blueprint
  - You only paint the changed wall
  - You don't rebuild the entire building

## Real Examples

### YouTube:

- You search → page changes
- But the search bar stays the same
- Only the content below changes

### Instagram:

- You have 4 chats with notifications
- A 5th message arrives
- Only that small section updates
- The rest of the page stays

## Benefits

- **Faster** — no unnecessary work
- **Efficient** — browser does less
- **Better UX** — app feels instantly responsive

# PART 8 — JSX (JavaScript XML)

## 8.1 What Is JSX?

JSX = JavaScript + XML (HTML-like syntax inside JS).

It's a combination of HTML and JavaScript.

## 8.2 Without JSX vs With JSX

Without JSX (plain JS):

javascript

```
const h1 = document.createElement("h1");  
h1.innerHTML = "Hello";  
document.body.appendChild(h1);
```

With JSX (React):

jsx

```
const h1 = <h1>Hello</h1>;
```

That's it. Much cleaner.

## 8.3 JSX Rules

1. Can't return two elements at once:

jsx

```
// ❌ Wrong
```

```
return (  
  <h1>Hello</h1>  
  <h2>World</h2>  
);
```

```
// ✅ Correct – wrap in a parent
```

```
return (  
  <div>  
    <h1>Hello</h1>  
    <h2>World</h2>  
  </div>  
);
```

```
    </div>  
  );
```

## 2. Use `className` instead of `class` :

jsx

```
<div className="parent">Hello</div>
```

Because `class` is a reserved keyword in JavaScript.

## 3. Use `htmlFor` instead of `for` :

jsx

```
<label htmlFor="name">Name</label>
```

## 4. Curly braces for JavaScript expressions:

jsx

```
const user = "Sarthak";  
return <h1>Hello {user}</h1>;
```

## 5. Self-closing tags allowed:

jsx

```
  
<input type="text" />  
<Card />
```

# 8.4 Calling Functions in JSX

jsx

```
const ABC = () => {  
  console.log("Hello");  
};
```

```
// Both work:
```

```
{ABC()}  
<ABC />
```

## PART 9 — CREATING A REACT PROJECT WITH VITE

### 9.1 Why Vite?

Previously, React used `create-react-app`. But now React recommends frameworks. For learning, we use **Vite** — a **bundler**.

#### What is a bundler?

- A tool that creates project folders and manages files
- Vite is fast and modern

### 9.2 Command

```
bash
```

```
npm create vite
```

#### Steps:

1. Enter project name (e.g., `001-folder` )
2. Select **React**
3. Select **JavaScript** (or TypeScript)
4. `cd project-name`
5. `npm install`
6. `npm run dev`

**Note:** Don't use `@latest` — it can cause errors. Use plain `npm create vite` .

### 9.3 Folder Structure

```
text
```

```
project/
├─ node_modules/      ← dependencies (don't touch)
├─ public/            ← static assets (images, fonts)
├─ src/
│   ├─ assets/        ← SVGs, frequently used images
│   ├─ App.jsx         ← main component
│   ├─ main.jsx        ← entry point
│   ├─ index.css       ← global styles
│   └─ App.css         ← App-specific styles
├─ index.html         ← HTML boilerplate
├─ package.json        ← config, dependencies
├─ vite.config.js      ← Vite config
├─ .eslintrc.js        ← ESLint rules
├─ .gitignore          ← files not to commit
└─ README.md           ← project description
```

## 9.4 The Rendering Flow

1. index.html contains:

```
html
```

```
<div id="root"></div>
```

2. main.jsx :

```
jsx
```

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';
```

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

3. App.jsx :

```
jsx
```

```
const App = () => {  
  return <h1>Hello</h1>;  
};  
export default App;
```

## Hierarchy:

text

```
index.html (grandparent)  
  → main.jsx (parent)  
    → App.jsx (child)  
      → other components
```

## 9.5 Configuration Files

`.eslintrc.js` — ESLint rules. Keeps code consistent across teams. Prevents conflicts like semicolons vs no-semicolons, single vs double quotes, etc.

`.gitignore` — files not to push to GitHub:

- `node_modules/`
- `.env` (API keys, secrets)
- `.vscode/`

**Why `.env` ?** Contains API keys. If pushed publicly, anyone can misuse your project.

`package.json` — project metadata:

- Dependencies
- Scripts
- Versions

`vite.config.js` — Vite plugins, configs.

## 9.6 VS Code Extensions

- **Simple React Snippets** — `rfce`, `rfc` shortcuts
- **ES7 React/Redux/GraphQL/React-Native snippets**
- **Tailwind CSS IntelliSense** — Tailwind class suggestions

# PART 10 — COMPONENT-BASED ARCHITECTURE

## 10.1 What Is a Component?

A **component** is a small, reusable piece of UI with a specific responsibility.

**Examples:**

- Navbar
- Card
- Button
- Footer
- Sidebar
- Feed post

## 10.2 Benefits

1. **Reusability** — Write once, use many times
2. **Isolation** — Change one without breaking others
3. **Performance** — Only that component re-renders
4. **Maintainability** — Easier to debug and update

## 10.3 Real Example — Sheriyans Website

If you break Sheriyans' website into components:

- Navbar = 1 component
- Logo = 1 component
- Nav links = 1 component
- Hero text = 1 component
- Button = 1 component
- Stats section = 1 component
- Each stat = 1 component
- Footer = 1 component

When you change the notification count:

- Only that component re-renders
- Not the whole page

## 10.4 Creating Components

**Step 1:** Create `components/` folder in `src/`

**Step 2:** Create `Card.jsx` :

```
jsx

const Card = () => {
  return (
    <div className="card">
      <h1>Hello</h1>
    </div>
  );
};

export default Card;
```

**Step 3:** Import and use in App:

```
jsx

import Card from './components/Card';

const App = () => {
  return (
    <div>
      <Card />
      <Card />
      <Card />
    </div>
  );
};
```

You get 3 cards, all identical.

## 10.5 Feature-Based vs Atomic Design

**Feature-Based:**

- Break by **features/pages**
- Entire navbar = 1 component
- Entire card section = 1 component
- Each page = 1 component

### Atomic Design:

- Break into **smallest pieces**
- Logo = separate component
- Each nav link = separate component
- Button = separate component
- Reuse everywhere

**Best Practice:** Combination of both.

## PART 11 — PROPS (Properties)

### 11.1 What Are Props?

Props are the data passed from a parent component to a child component.

#### Key Rules:

- Always flow **top** → **bottom** (parent → child)
- **Never** flow bottom → top
- Received as an **object**

### 11.2 Passing Props

jsx

```
<Card user="Sarthak" age={21} image="url.jpg" />
```

#### Note:

- Strings: user="Sarthak" (direct quotes)

- Numbers/Arrays/Objects: `age={21}` (curly braces)

## 11.3 Receiving Props

jsx

```
const Card = (props) => {  
  console.log(props); // { user: "Sarthak", age: 21, image: "url.jpg" }  
  return (  
    <div>  
      <h1>{props.user}</h1>  
      <p>{props.age}</p>  
    </div>  
  );  
};
```

## 11.4 Renaming Props

You can rename to anything:

jsx

```
const Card = (chacha) => {  
  return <h1>{chacha.user}</h1>;  
};
```

props is just a convention. chacha , raju , anything works.

## 11.5 Destructuring Props

jsx

```
const Card = ({ user, age, image }) => {  
  return (  
    <div>  
      <h1>{user}</h1>  
      <p>{age}</p>  
      <img src={image} />  
    </div>  
  );  
};
```

## 11.6 Props Drilling

Passing props through multiple levels:

```
text
```

```
App → Section1 → Page1Content → RightContent → Card
```

**Problem:** Middle components don't need the data, but must pass it anyway.

**Solution:** Context API (later).

## 11.7 Analogy — Function Arguments

Props are like function arguments:

```
javascript
```

```
function ABC(a) {  
  console.log("Hello", a);  
}  
ABC(10); // Hello 10
```

Same with components:

```
jsx
```

```
<Card user="Sarthak" />  
// Inside Card: props = { user: "Sarthak" }
```

## 11.8 Practical Example — Cards Project

**users.js:**

```
javascript
```

```
export const users = [  
  {  
    id: 1,  
    image: "url1",  
    intro: "Sarthak",  
    tag: "Satisfied"  
  },  
]
```

```
{
  id: 2,
  image: "url2",
  intro: "Harsh",
  tag: "Underbanked"
},
{
  id: 3,
  image: "url3",
  intro: "Aman",
  tag: "Underserved"
}
];
```

### App.jsx:

```
jsx

import { users } from './users';

const App = () => {
  return (
    <div>
      <Section1 users={users} />
    </div>
  );
};
```

### RightContent.jsx:

```
jsx

const RightContent = (props) => {
  return (
    <div>
      {props.users.map((user, idx) => (
        <Card key={idx} user={user} />
      ))}
    </div>
  );
};
```

### Card.jsx:

```
jsx
```

```
const Card = (props) => {  
  return (  
    <div>  
      <img src={props.user.image} />  
      <h1>{props.user.intro}</h1>  
      <span>{props.user.tag}</span>  
    </div>  
  );  
};
```

## Key Takeaways:

- Always pass a unique `key` in `map`
- Better to wrap the component in a `div` and pass `key` to the `div`
- React uses `key` to identify which items changed

# PART 12 — MAP METHOD FOR ARRAYS

## 12.1 Why `map` ?

`map` is a JavaScript method that transforms each element in an array.

javascript

```
const arr = [10, 20, 30];  
const doubled = arr.map(x => x * 2);  
// [20, 40, 60]
```

## 12.2 Using `map` in JSX

jsx

```
const users = ["Sarthak", "Harsh", "Aman"];  
  
return (  
  <div>  
    {users.map((user, idx) => (  
      <h1 key={idx}>{user}</h1>  
    ))}  
  </div>  
)
```

```
    </div>  
  );
```

## Why key ?

- React needs to identify each element uniquely
- Like roll numbers in a class
- Without key, React re-renders everything

## 12.3 Map With Array of Objects

jsx

```
const users = [  
  { name: "Sarthak", age: 30, image: "url1" },  
  { name: "Harsh", age: 36, image: "url2" },  
  { name: "Aman", age: 20, image: "url3" }  
];  
  
return (  
  <div>  
    {users.map((user, idx) => (  
      <div key={idx}>  
        <img src={user.image} />  
        <h1>{user.name}</h1>  
        <p>{user.age}</p>  
      </div>  
    ))}  
  </div>  
);
```

## 12.4 The Key Warning

If you don't pass key , React shows:

"Each child in a list should have a unique 'key' prop."

### Best Practice:

jsx

```
<div key={idx}>  
  <Card user={user} />  
</div>
```

```
</div>
```

Don't pass `key` directly to your custom component — wrap it.

## PART 13 — CSS IN REACT

### 13.1 Types of CSS in React

1. **Global CSS** — `index.css`
2. **Component-Specific CSS** — `Button.css` , `Header.css`
3. **CSS Modules** — `Button.module.css`
4. **Tailwind CSS** — Utility-first
5. **Styled Components / Emotion** — CSS-in-JS (not covered)

### 13.2 Global CSS

css

```
/* index.css */
body {
  background: #111;
  color: #fff;
}
```

Imported in `main.jsx` :

jsx

```
import './index.css';
```

**Good for:** Small projects.

**Bad for:** Large projects (hard to manage).

### 13.3 Component-Specific CSS

text

```
components/  
├─ Button/  
│   ├─ Button.jsx  
│   └─ Button.css  
└─ Header/  
    ├─ Header.jsx  
    └─ Header.css
```

### Button.jsx:

```
jsx  
  
import './Button.css';  
  
const Button = () => {  
  return <button className="btn">Click</button>;  
};
```

**Problem:** Class name collisions. If both Button and Header use `.btn`, they conflict.

## 13.4 CSS Modules

Rename to `Button.module.css` :

```
css  
  
/* Button.module.css */  
.btn {  
  background: red;  
  padding: 10px 20px;  
}
```

### Button.jsx:

```
jsx  
  
import styles from './Button.module.css';  
  
const Button = () => {  
  return <button className={styles.btn}>Click</button>;  
};
```

### Benefits:

- Classes are **hashed** (e.g., `_btn_x8d3a` )
- **No collisions**
- Only the component can access its classes

## 13.5 Tailwind CSS (Utility-First)

### What is it?

- Not a component library
- **Utility-first** library
- Small classes for every CSS property

### Examples:

- `bg-red-600` — background red
- `text-white` — white text
- `px-5` — padding left/right
- `py-2` — padding top/bottom
- `rounded-lg` — rounded corners
- `flex` — `display: flex`
- `items-center` — `align-items: center`
- `justify-between` — `justify-content: space-between`

### Installation:

```
bash
```

```
npm install tailwindcss @tailwindcss/vite
```

### vite.config.js:

```
javascript
```

```
import tailwindcss from '@tailwindcss/vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react(), tailwindcss()]
});
```

## index.css:

CSS

```
@import "tailwindcss";
```

## Usage:

jsx

```
<h1 className="bg-red-600 text-white px-5 py-2 rounded-lg">  
  Hello  
</h1>
```

**Benefit:** No CSS files, no class name collisions, fast prototyping.

**VS Code Extension:** Tailwind CSS IntelliSense for autocomplete.

## 13.6 The Bootstrap Digression

The instructor jokes:

"Bootstrap was used during Raja Akbar's time. It's old. Learn Tailwind instead. But don't write Bootstrap in your resume — people will think you're outdated."

Funny story aside, the point: **Tailwind is modern.**

## PART 14 — FEATURE-BASED VS ATOMIC DESIGN

### 14.1 Feature-Based Design

Break the app by **features**:

- Landing Page = 1 feature = 1 component
- Navbar = 1 feature = 1 component
- Card section = 1 feature = 1 component
- Footer = 1 feature = 1 component

## 14.2 Atomic Design

Break into **smallest pieces**:

- Button = 1 component
- Heading = 1 component
- Logo = 1 component
- Nav link = 1 component
- Each reusable everywhere

## 14.3 Analogy

**Feature-Based:** You buy a pre-built room.

**Atomic Design:** You build the room from individual bricks.

## 14.4 Best Practice

**Combination.** Use feature-based for structure, atomic for reusable pieces.

text

```
components/  
├─ Section1/  
│   ├─ Section1.jsx  
│   ├─ Navbar.jsx  
│   ├─ Page1Content.jsx  
│   └─ LeftContent/  
│       ├─ LeftContent.jsx  
│       ├─ HeroText.jsx  
│       └─ Arrow.jsx  
└─ RightContent/  
    ├─ RightContent.jsx  
    └─ RightCard/  
        ├─ RightCard.jsx  
        └─ RightCardContent.jsx  
└─ Section2/  
    └─ Section2.jsx
```

# PART 15 — FUNCTION CALLING IN REACT (EVENT HANDLING)

## 15.1 Basic Event Handling

```
jsx

const btnClicked = () => {
  console.log("Button clicked");
};

<button onClick={btnClicked}>Click</button>
```

**Important:** Don't write `onClick={btnClicked()}` — that calls immediately on render.

## 15.2 With Parameters

```
jsx

<button onClick={() => btnClicked("param")}>Click</button>
```

## 15.3 Common Events

| Event         | Trigger      |
|---------------|--------------|
| onClick       | Click        |
| onDoubleClick | Double click |
| onMouseEnter  | Mouse enter  |
| onMouseLeave  | Mouse leave  |
| onMouseMove   | Mouse move   |
| onChange      | Input change |
| onSubmit      | Form submit  |
| onScroll      | Scroll       |

| Event   | Trigger     |
|---------|-------------|
| onWheel | Mouse wheel |

## 15.4 Event Object

```
jsx

const handleChange = (e) => {
  console.log(e.target.value); // Current input value
  console.log(e.target);      // The target element
};
```

## 15.5 Prevent Default

```
jsx

const handleSubmit = (e) => {
  e.preventDefault(); // Prevents page reload
};
```

## 15.6 Real Example — Input Tracking

```
jsx

const [title, setTitle] = useState("");

<input
  value={title}
  onChange={(e) => setTitle(e.target.value)}
/>
```

Every keystroke updates `title`, which updates the input.

## 15.7 Mouse Movement Example

```
jsx

<div onMouseMove={(e) => console.log(e.clientX, e.clientY)}>
  Move your mouse
</div>
```

```
</div>
```

- `e.clientX` — horizontal position
- `e.clientY` — vertical position

## 15.8 Scroll Example

```
jsx
```

```
<div onWheel={(e) => console.log(e.deltaY)}>  
  Scroll me  
</div>
```

- `e.deltaY` — scroll speed and direction
- Positive = scrolling down
- Negative = scrolling up

## PART 16 — HOOKS: INTRODUCTION

### 16.1 What Are Hooks?

Hooks are **special functions** in React that let you "hook into" React features:

- State management
- Side effects
- Context
- Refs
- Performance optimization

### 16.2 Why Hooks?

Before hooks, React used **class components** for stateful logic. But class components are now obsolete.

Hooks make everything possible in **functional components**.

## 16.3 The Hooks List

| Hook        | Purpose                       |
|-------------|-------------------------------|
| useState    | Manage component state        |
| useEffect   | Handle side effects           |
| useContext  | Consume context               |
| useReducer  | Complex state management      |
| useRef      | Reference DOM, mutable values |
| useMemo     | Memoize values                |
| useCallback | Memoize functions             |

## PART 17 — USESTATE HOOK

### 17.1 The Problem Without useState

```
jsx

let a = 20;

const changeA = () => {
  a = 30;
  console.log(a); // 30 (but UI doesn't update)
};

<h1>Value of a is {a}</h1>
<button onClick={changeA}>Click</button>
```

**Problem:** The UI doesn't update even though `a` changed.

### 17.2 Why?

React doesn't watch variables. It re-renders only when state changes.

Direct DOM manipulation is **not allowed**. You tell React to update.

## 17.3 The Solution — useState

jsx

```
import { useState } from 'react';

const [a, setA] = useState(20);

const changeA = () => {
  setA(30); // React updates UI
};

<h1>Value of a is {a}</h1>
<button onClick={changeA}>Click</button>
```

## 17.4 Syntax

jsx

```
const [value, setValue] = useState(initialValue);
```

- value — current state (read-only)
- setValue — function to update
- initialValue — starting value

## 17.5 Counter Example

jsx

```
const [num, setNum] = useState(0);

const increase = () => setNum(num + 1);
const decrease = () => setNum(num - 1);
const jumpFive = () => setNum(num + 5);

<button onClick={increase}>Increase</button>
<button onClick={decrease}>Decrease</button>
<button onClick={jumpFive}>Jump by 5</button>
<h1>{num}</h1>
```

## 17.6 Asynchronous Nature

jsx

```
const handleClick = () => {  
  setNum(30);  
  console.log(num); // Still old value!  
};
```

**Why?** State updates are async. React batches them for performance.

**Why not direct mutation?**

jsx

```
num = 30; // ❌ Not allowed – num is const
```

## 17.7 Updating Objects

**Wrong:**

jsx

```
const [user, setUser] = useState({ name: "Sarthak", age: 20 });  
user.name = "Aman"; // ❌ No re-render
```

**Correct:**

jsx

```
const newUser = { ...user, name: "Aman" };  
setUser(newUser);
```

**Why?** Arrays and objects are **reference types**. Mutating the same reference doesn't trigger re-render.

**Analogy:**

- Imagine a barber shop
- You and Adarsh both get haircuts from the same barber
- Adarsh gets a bad haircut, hits the barber

- Now you can't get a haircut either (shared reference)

### For Arrays:

jsx

```
const newArr = [...arr, 99];  
setArr(newArr);
```

## 17.8 Functional Updates

### Wrong (only increments by 1):

jsx

```
setNum(num + 1);  
setNum(num + 1);  
setNum(num + 1);
```

### Correct (increments by 3):

jsx

```
setNum(prev => prev + 1);  
setNum(prev => prev + 1);  
setNum(prev => prev + 1);
```

**Why?** React batches updates. Functional updates see the latest value.

## 17.9 Two-Way Binding

jsx

```
const [title, setTitle] = useState("");  
  
<input  
  value={title}  
  onChange={(e) => setTitle(e.target.value)}  
>
```

### What happens:

#### 1. User types "S"

2. `onChange` fires with `e.target.value = "S"`
3. `setTitle("S")` updates state
4. State updates input's value to "S"

**Two-way binding** = state ↔ input, both connected.

## 17.10 Form Submission

jsx

```
const submitHandler = (e) => {  
  e.preventDefault(); // Prevents reload  
  console.log(title, details);  
  setTitle("");  
  setDetails("");  
};  
  
<form onSubmit={submitHandler}>  
  <input value={title} onChange={(e) => setTitle(e.target.value)} />  
  <input value={details} onChange={(e) => setDetails(e.target.value)} />  
  <button>Add Note</button>  
</form>
```

## PART 18 — LOCAL STORAGE

### 18.1 What Is Local Storage?

Browser's memory.

- Stores data as **key-value pairs**
- Value must be a **string**
- Data persists even after closing the browser

**Analogy:**

- Like a small drawer in the browser
- You put things in, take things out
- It stays even when the browser is closed

## 18.2 Session Storage vs Local Storage

| Local Storage             | Session Storage               |
|---------------------------|-------------------------------|
| Persists forever          | Persists until browser closes |
| Shared across tabs        | Tab-specific                  |
| Larger capacity (~5-10MB) | Smaller capacity              |

## 18.3 Methods

javascript

```
// Set
localStorage.setItem("user", "Sarthak");

// Get
const user = localStorage.getItem("user"); // "Sarthak"

// Remove specific
localStorage.removeItem("user");

// Clear all
localStorage.clear();
```

## 18.4 Storing Objects/Arrays

javascript

```
// Store
const user = { name: "Sarthak", age: 30 };
localStorage.setItem("user", JSON.stringify(user));

// Retrieve
const stored = JSON.parse(localStorage.getItem("user"));
console.log(stored.name); // "Sarthak"
```

### Why JSON?

- Local storage only accepts strings
- `JSON.stringify` converts object → string

- `JSON.parse` converts string → object

## 18.5 Real Example — Instagram Login

When you log in:

1. Backend sends **access token** and **refresh token**
2. Stored in **cookies** (in local storage)
3. **Refresh token** expires in 15 minutes
4. **Access token** expires in 2 months
5. On each reload, tokens are checked → auto-login

If you delete those tokens from local storage → logged out.

## 18.6 Practical Use

- Theme preference (dark/light)
- Auth tokens
- User preferences
- Shopping cart
- Recently viewed

# PART 19 — API HANDLING

## 19.1 The Concept

**Frontend** = Restaurant interior (what user sees)

**Backend** = Kitchen (where food is prepared)

**API** = Waiter (connects both)

**Frontend asks backend:** "Give me data"

**Backend responds:** "Here's your data"

## 19.2 HTTP Methods

| Method      | Purpose            |
|-------------|--------------------|
| GET         | Fetch data         |
| POST        | Send data (create) |
| PUT / PATCH | Update data        |
| DELETE      | Delete data        |

## 19.3 Why APIs?

- Frontend doesn't have a database
- Data lives on the backend
- API is the bridge

## 19.4 Fetch API

jsx

```
const getData = async () => {  
  const response = await fetch("https://api.example.com/todos/1");  
  const data = await response.json();  
  console.log(data);  
};
```

### Steps:

1. `fetch()` returns a **Promise**
2. `await` waits for the response
3. `response.json()` returns another Promise
4. `await` again to get actual data

## 19.5 Axios (Preferred)

bash

```
npm install axios
```

jsx

```
import axios from "axios";

const getData = async () => {
  const response = await axios.get("https://api.example.com/users");
  console.log(response.data); // Data directly
};
```

## Why prefer Axios?

- Cleaner syntax
- Better error handling
- Interceptors for auth
- Request/response transformation

## 19.6 JSON

JSON = JavaScript Object Notation

json

```
[
  {
    "id": 1,
    "name": "Sarthak",
    "age": 30
  },
  {
    "id": 2,
    "name": "Harsh",
    "age": 36
  }
]
```

Every API response is typically JSON. Learn to read it.

## 19.7 Storing API Data in State

jsx

```

const [userData, setUserData] = useState([]);

const getData = async () => {
  const response = await axios.get("https://api.example.com/users");
  setUserData(response.data);
};

useEffect(() => {
  getData();
}, []);

return (
  <div>
    {userData.map((user, idx) => (
      <div key={idx}>
        <img src={user.image} />
        <h2>{user.name}</h2>
      </div>
    ))}
  </div>
);

```

## 19.8 Real API Examples

### JSONPlaceholder — fake API for testing

- <https://jsonplaceholder.typicode.com/users>
- <https://jsonplaceholder.typicode.com/posts>

### Lorem Picsum — random images

- <https://picsum.photos/v2/list?page=1&limit=10>

#### Parameters:

- `?page=1` — page number
- `&limit=10` — items per page
- `&id=5` — specific item

## 19.9 Pagination Example

jsx

```
const [index, setIndex] = useState(1);
const [data, setData] = useState([]);

const getData = async () => {
  const response = await axios.get(
    `https://picsum.photos/v2/list?page=${index}&limit=10`
  );
  setData(response.data);
};

const prev = () => {
  if (index > 1) setIndex(index - 1);
};

const next = () => setIndex(index + 1);
```

## 19.10 Loading State

jsx

```
const [loading, setLoading] = useState(false);

const getData = async () => {
  setLoading(true);
  setData([]); // Clear old data
  const response = await axios.get("...");
  setData(response.data);
  setLoading(false);
};

return (
  <div>
    {loading ? <h3>Loading...</h3> : userData.map(...)}
  </div>
);
```

## 19.11 Infinite Scrolling

- Load more as user scrolls
- Trigger API call on scroll
- Append new data to existing

jsx

```
const [data, setData] = useState([]);
const [page, setPage] = useState(1);

useEffect(() => {
  const handleScroll = () => {
    if (window.innerHeight + window.scrollY >= document.body.offsetHeight - 10) {
      setPage(page + 1);
    }
  };
  window.addEventListener("scroll", handleScroll);
  return () => window.removeEventListener("scroll", handleScroll);
}, [page]);

useEffect(() => {
  axios.get(`...?page=${page}`).then(res => {
    setData(prev => [...prev, ...res.data]);
  });
}, [page]);
```

## PART 20 — USEEFFECT HOOK

### 20.1 What Is useEffect?

`useEffect` handles **side effects** in React.

**Side effects include:**

- API calls
- DOM manipulation
- Event listeners
- Timers (`setTimeout`, `setInterval`)
- Local storage sync

### 20.2 The Analogy — Main Road vs Side Lane

Imagine a **big road** with a **big truck**.

- **Truck** = React's rendering process

- **Big truck** = takes up the entire road
- You want to ride alongside on your **bicycle**
- But the truck covers the whole road
- **Solution:** Use the **side lane** (footpath)
- The truck keeps moving on its path
- You ride on the side lane
- **Neither disturbs the other**

`useEffect` is the **side lane** for side effects.

## 20.3 Syntax

jsx

```
useEffect(() => {  
  // Side effect code  
}, [dependencies]);
```

## 20.4 Three Modes

### Mode 1 — Runs Every Render

jsx

```
useEffect(() => {  
  console.log("Runs every time");  
});
```

**Warning:** Usually wrong usage.

### Mode 2 — Runs Once (Mount)

jsx

```
useEffect(() => {  
  console.log("Runs once on mount");  
}, []);
```

**Most common:** API calls on page load.

## Mode 3 — Runs When Dependency Changes

jsx

```
useEffect(() => {  
  console.log("A changed");  
}, [a]);
```

**Runs when** `a` **changes.**

## 20.5 Real Example

jsx

```
const [a, setA] = useState(0);  
const [b, setB] = useState(0);  
  
const aChanging = () => console.log("A changed");  
const bChanging = () => console.log("B changed");  
  
useEffect(() => {  
  aChanging();  
}, [a]); // Only runs when A changes  
  
useEffect(() => {  
  bChanging();  
}, [b]); // Only runs when B changes
```

## 20.6 Cleanup Function

jsx

```
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log("Tick");  
  }, 1000);  
  
  return () => {  
    clearInterval(timer); // Cleanup on unmount  
  };  
}, []);
```

```
};  
}, []);
```

## 20.7 Real API Example

jsx

```
const [userData, setUserData] = useState([]);  
  
const getData = async () => {  
  const response = await axios.get("https://picsum.photos/v2/list");  
  setUserData(response.data);  
};  
  
useEffect(() => {  
  getData();  
}, []); // Runs once on mount
```

## 20.8 Dependency Array — Multiple Values

jsx

```
useEffect(() => {  
  // Runs when a OR b changes  
}, [a, b]);
```

## 20.9 Common Pitfalls

### StrictMode double render:

In development, React StrictMode renders components twice. Your `useEffect` will run twice. This is intentional. Remove StrictMode to see single runs.

## PART 21 — REACT ROUTER DOM

### 21.1 What Is Routing?

**Routing** = Navigating between different pages/views.

**Example:**

- `sharans.com/` → Home
- `sharans.com/courses` → Courses
- `sharans.com/about` → About
- `sharans.com/contact` → Contact

## 21.2 SPA (Single Page Application)

**What is SPA?**

- A website that **doesn't reload** on navigation
- URL changes but the page doesn't refresh
- Feels like a native app

**Example:**

- YouTube — when you click a video, URL changes but no reload
- Instagram — same
- Sheriyans — same

**How?**

React Router intercepts link clicks and updates the view without reloading.

## 21.3 Installation

```
bash
```

```
npm install react-router-dom
```

## 21.4 Types of Routers

| Router        | Use Case                     |
|---------------|------------------------------|
| BrowserRouter | Most common (SPA)            |
| HashRouter    | Static sites (uses # in URL) |

| Router       | Use Case              |
|--------------|-----------------------|
| MemoryRouter | React Native, testing |
| StaticRouter | Next.js, Remix (SSR)  |

We use **BrowserRouter** 99% of the time.

## 21.5 Basic Setup

main.jsx:

```
jsx

import { BrowserRouter } from 'react-router-dom';

<BrowserRouter>
  <App />
</BrowserRouter>
```

App.jsx:

```
jsx

import { Routes, Route } from 'react-router-dom';
import Home from './pages/Home';
import About from './pages/About';
import Contact from './pages/Contact';

<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
  <Route path="/contact" element={<Contact />} />
</Routes>
```

## 21.6 Navigation with Link

```
jsx

import { Link } from 'react-router-dom';

<nav>
  <Link to="/">Home</Link>
```

```

    <Link to="/about">About</Link>
    <Link to="/contact">Contact</Link>
  </nav>

```

## Why <Link> and not <a> ?

- <a> causes full page reload
- <Link> updates URL without reload
- Uses HTML5 History API

## 21.7 Not Found Page

jsx

```

<Route path="*" element={<NotFound />} />

```

Any unmatched URL shows NotFound .

## 21.8 Nested Routes

jsx

```

<Route path="/product" element={<Product />}>
  <Route path="men" element={<Men />} />
  <Route path="women" element={<Women />} />
</Route>

```

### Product.jsx:

jsx

```

import { Outlet } from 'react-router-dom';

const Product = () => {
  return (
    <div>
      <h1>Product Page</h1>
      <Outlet /> { /* Child routes render here */ }
    </div>
  );
};

```

**Important:** In nested routes, paths are **relative** (no leading `/`).

## 21.9 Dynamic Routes

jsx

```
<Route path="/courses/:courseId" element={<CourseDetail />} />
```

**CourseDetail.jsx:**

jsx

```
import { useParams } from 'react-router-dom';
```

```
const { courseId } = useParams();  
console.log(courseId); // Value from URL
```

**Examples:**

- `/courses/frontend` → `courseId = "frontend"`
- `/courses/backend` → `courseId = "backend"`
- `/courses/3js` → `courseId = "3js"`

Any value works. It's **dynamic**.

## 21.10 useNavigate Hook

jsx

```
import { useNavigate } from 'react-router-dom';
```

```
const navigate = useNavigate();
```

```
// Go to home  
navigate("/");
```

```
// Go back  
navigate(-1);
```

```
// Go forward  
navigate(1);
```

### Use cases:

- After login → navigate to dashboard
- After logout → navigate to home
- After form submission → navigate to success page

## 21.11 NavLink

jsx

```
import { NavLink } from 'react-router-dom';

<NavLink
  to="/about"
  className={({ isActive }) => isActive ? "active" : ""}
>
  About
</NavLink>
```

Shows an "active" style when the route matches.

## PART 22 — CONTEXT API

### 22.1 Problem — Props Drilling

Data flows **top** → **bottom** only.

text

App → Navbar → Sidebar → UserProfile → Username

Every level must pass the prop, even if they don't need it.

#### Analogy — Netflix Password:

- Deepak buys a Netflix subscription
- Passes to Vedant
- Vedant passes to Harsh
- If Deepak changes password, everyone must be told

- **Painful.**

**Better:** Write the password on a **wall**. Everyone can see it.

## 22.2 Data Centralization

Context API **centralizes data**.

Every component can:

- **Access** the data
- **Change** the data (if allowed)

## 22.3 Three Steps

1. **Create Context**
2. **Provide Data**
3. **Use Data**

## 22.4 Step 1 — Create Context

```
jsx

// context/ThemeContext.jsx
import { createContext } from 'react';
export const ThemeDataContext = createContext();
```

## 22.5 Step 2 — Provide Data

```
jsx

// App.jsx
import { ThemeDataContext } from './context/ThemeContext';
import { useState } from 'react';

const App = () => {
  const [theme, setTheme] = useState("light");

  return (
    <ThemeDataContext.Provider value={[theme, setTheme]}>
      <Navbar />
    </ThemeDataContext.Provider>
  );
};
```

```

    </ThemeDataContext.Provider>
  );
};

```

## 22.6 Step 3 — Use Data

jsx

```

// Button.jsx
import { useContext } from 'react';
import { ThemeDataContext } from '../context/ThemeContext';

const Button = () => {
  const [theme, setTheme] = useContext(ThemeDataContext);

  return (
    <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
      Change Theme
    </button>
  );
};

```

## 22.7 Children Prop

jsx

```

const Navbar = (props) => {
  return <div>{props.children}</div>;
};

<Navbar>
  <h2>Hello</h2>
  <p>World</p>
</Navbar>

```

`props.children` contains the elements between `<Navbar>` and `</Navbar>`.

### With destructuring:

jsx

```

const Navbar = ({ children }) => {
  return <div>{children}</div>;
};

```

```
};
```

## 22.8 Context in Larger Apps

For Instagram-like apps, you'd have multiple contexts:

- `UserContext` — user data
- `StoryContext` — stories
- `PostContext` — posts
- `MessageContext` — messages

Each context handles one domain.

## 22.9 Context Provider and Consumer

**Provider** — gives the data:

```
jsx
```

```
<ThemeDataContext.Provider value={...}>
```

**Consumer** — reads the data:

```
jsx
```

```
const data = useContext(ThemeDataContext);
```

## PART 23 — PROJECTS BUILT IN THE COURSE

### Project 1 — Cards

- Props drilling
- Component-based architecture
- Array of objects with `map`

### Project 2 — Job Listings

- Array of objects
- 10 companies with data
- Map through array
- Dynamic rendering

## **Project 3 — Website UI**

- Digital banking platform
- Section 1 + Section 2
- Complex nested components
- Props through multiple levels
- Realistic UI

## **Project 4 — Notes App**

- Form handling
- Two-way binding
- Local storage integration
- Delete functionality
- Component separation

## **Project 5 — Gallery Application**

- API handling
- Axios
- Pagination (Prev / Next)
- Loading state
- Dynamic routes

# **PART 24 — KEY PATTERNS & BEST PRACTICES**

## **24.1 Folder Structure**

text

```
src/  
├─ components/  
│   ├─ Section1/  
│   │   ├─ Section1.jsx  
│   │   ├─ Navbar.jsx  
│   │   └─ Page1Content.jsx  
│   └─ ...  
└─ Section2/  
├─ pages/  
│   ├─ Home.jsx  
│   └─ About.jsx  
└─ Contact.jsx  
├─ context/  
│   └─ ThemeContext.jsx  
├─ assets/  
├─ App.jsx  
├─ main.jsx  
└─ index.css
```

## 24.2 Naming Conventions

- Components: **PascalCase** ( Card.jsx , Navbar.jsx )
- Functions: **camelCase** ( handleClick , getData )
- CSS classes: **kebab-case** ( nav-bar , card-content )
- Constants: **UPPER\_SNAKE\_CASE** ( API\_URL )

## 24.3 Component Best Practices

- Small, focused components
- Props in, JSX out
- Use key in lists
- Destructure props
- Avoid inline functions in JSX (useCallback)
- Keep state minimal
- Lift state up when needed

## 24.4 useState Best Practices

- Don't mutate state directly
- Use functional updates when needed
- Group related state
- Keep state local when possible

## 24.5 useEffect Best Practices

- Always specify dependencies
- Clean up subscriptions
- Don't use for derived state
- Async functions inside, not outside

## PART 25 — COMMON PITFALLS

1. **Mutating state directly**
2. **Forgetting** `key` **in lists**
3. **Calling functions immediately:** `onClick={fn()}` VS `onClick={fn}`
4. **Not handling async state updates**
5. **Overusing** `useEffect`
6. **Prop drilling**
7. **Re-rendering too often**
8. **Missing dependencies in** `useEffect`
9. **Using** `index` **as key when list changes**
10. **Direct DOM manipulation**

## PART 26 — INSTRUCTOR'S PHILOSOPHY

Some quotes and ideas from the transcript:

"React JS is basically a JavaScript library which is used to make UI."

"Props data always flows from top to bottom, never bottom to top."

"useEffect is one of the most powerful hooks but also one of the most confusing."

"Don't mutate state directly — always use the setter function."

"The best way to learn is by building projects."

"What is your 49.5?" — Referencing the movie *Bhaag Milkha Bhaag*, asking viewers about their ultimate life goal.

"Doctors won't read this, only those who truly want to learn will."

He emphasizes:

- **Practice over theory**
- **Build projects**
- **Watch chapters one at a time**
- **Don't skip**
- **Ask questions in comments**

## PART 27 — SUMMARY TABLE

| Topic     | Key Idea                  |
|-----------|---------------------------|
| React     | JS library for UIs        |
| Component | Reusable UI piece         |
| Props     | Parent → child data       |
| State     | Component's internal data |
| useState  | Manage state              |
| useEffect | Handle side effects       |
| Context   | Global data               |
| Router    | Navigation                |
| API       | Fetch data from backend   |

| Topic           | Key Idea          |
|-----------------|-------------------|
| Virtual DOM     | Efficient updates |
| JSX             | HTML + JS         |
| Local Storage   | Browser memory    |
| Two-Way Binding | State ↔ Input     |
| Pagination      | Page-by-page data |
| Tailwind        | Utility-first CSS |
| CSS Modules     | Scoped CSS        |

## PART 28 — FINAL THOUGHTS

This course is **massive** (the video itself is many hours). The transcript is essentially a **full React course**.

### What you get:

- React fundamentals
- Hooks in depth
- Routing
- Context API
- API handling
- Real projects
- Best practices
- Common pitfalls

### What to do next:

1. Watch each chapter
2. **Code along** — don't just watch
3. Build the projects

4. Practice with your own ideas
5. Ask questions in comments
6. Build even bigger apps
7. Consider advanced topics: Redux, TypeScript, Next.js

### The instructor's promise:

"If you follow this video, build the projects, and practice — you will learn React. This is my guarantee."

## CLOSING

This transcript is a **complete React JS course** covering:

- What is React and why it exists
- JavaScript prerequisites
- Development environment setup
- Core concepts: components, props, state
- Hooks: useState, useEffect, useContext
- Routing with React Router
- API integration
- Local storage
- CSS in React
- Real-world projects

### The most important lesson: Practice.

React is learned by building. Watch the video, then **open VS Code and start coding**. Build the projects. Break things. Fix them. Repeat.

"Main tha aapka instructor Sarthak Sharma. Aap dekh rahe hain Sheriyans Coding School. Thank you so much for watching this video."

If you want, I can:

- **Dive deeper** into any single topic (e.g., useEffect with 20 examples)
- **Convert this into notes** for quick revision
- **Write code examples** for each concept
- **Create a learning roadmap** based on this course
- **Explain any specific project** from the course step-by-step

Just tell me which direction you'd like to go.